BLOQUE III - TEMA 7

DESARROLLO DE SISTEMAS

Aplicaciones web. Desarrollo web front-end y en servidor, multiplataforma y multidispositivo. Lenguajes: HTML, XML y sus derivaciones. Navegadores y lenguajes de programación web. Lenguajes de script.

Supera. by thePower

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2024.

ÍNDICE

1. INTRODUCCIÓN APLICACIONES WEB 5
1.1 Evolución de la web: de páginas estáticas a aplicaciones interactivas 5
1.2 Arquitectura cliente-servidor 6
2. DESARROLLO WEB FRONT-END Y EN SERVIDOR 7
2.1 Tecnologías y frameworks front-end 7
2.1.1 Lenguajes principales 7
2.1.2 Frameworks y bibliotecas más populares 8
2.1.3 Herramientas complementarias. 8
2.2 Tecnologías y frameworks de desarrollo en servidor 8
2.2.1 Arquitecturas y paradigmas 9
2.2.2 Bases de datos y almacenamiento 9
3. DESARROLLO WEB MULTIPLATAFORMA Y MULTIDISPOSITIVO 9
3.1 Responsive Web Design (Diseño Web Adaptativo) 9
3.2 Progressive Web Apps (PWAs) 10
3.3 Frameworks y herramientas para el desarrollo multiplataforma 11
3.4 Android y sus versiones 12
4. HTML, XML Y SUS DERIVACIONES 13
4.1 HTML 13
4.2 HTML5 17
4.3 CSS (Cascading Style Sheets) 19
4.3.1. Orden de los selectores en CSS 20
4.4 Grid Layout en CSS 23
4.5 Flexbox (Flexible Box Layout) 24
4.6 Bootstrap: Framework de diseño web responsivo 25

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025.

5. XML: CARACTERÍSTICAS Y USOS 27

6. LENGUAJES DE SCRIPT. JAVASCRIPT 30

6.1 Características principales de JavaScript 30

6.2 Eventos comunes en HTML (clasificados por tipo) 31

6.3 Extracción de elementos del DOM en HTML con JavaScript 32

6.4. JSON 33

6.5 Frameworks y bibliotecas de JavaScript 35

6.6 AJAX 35

7. LENGUAJES DE SCRIPT. PHP 38

8. LENGUAJE DE PROGRAMACIÓN PYTHON 44

8.1 Sintaxis y datos 44

8.2 Variables y constantes 45

8.3 Operadores en Python 46

8.4 Control de flujo en Python 48

8.5 Funciones en Python 48

8.6 Estructuras de datos en Python 49

8.7 Cadenas de texto (strings) en Python 50

8.8 Frameworks en Python 51

9. LENGUAJE DE PROGRAMACIÓN RUBY 52

9.1 Frameworks en Ruby 52

Significado de los iconos que aparecen dentro de los TEMAS:



Examen


Importante


Recordatorio


Atención

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025.

1. INTRODUCCIÓN APLICACIONES WEB

Las **aplicaciones web** son programas accesibles a través de un navegador y ejecutados en una arquitectura distribuida, que permiten a los usuarios realizar tareas complejas sin necesidad de instalar software local.

Una aplicación web se diferencia de una simple página web estática en que permite la **interacción con el usuario**, el **procesamiento dinámico de datos**, la **persistencia de información en bases de datos** y la integración con otros sistemas o servicios (como pasarelas de pago, servicios de autenticación o APIs externas).

1.1 EVOLUCIÓN DE LA WEB: DE PÁGINAS ESTÁTICAS A APLICACIONES INTERACTIVAS

Desde su aparición a comienzos de los años 90, la Web ha experimentado una transformación profunda en cuanto a capacidades técnicas, nivel de interactividad y complejidad de los servicios ofrecidos. Esta evolución se puede agrupar en varias etapas clave:

Web 1.0 – Web estática (1990s)

- Se caracteriza por páginas web **escritas en HTML estático**, sin interacción ni personalización.
- Los sitios servían contenido fijo, diseñado para la consulta unidireccional de información.
- Los cambios requerían modificar manualmente los archivos HTML y volver a subirlos al servidor.

Web 2.0 – Web dinámica e interactiva (2000s)

- Introducción de lenguajes de programación en el servidor (PHP, ASP, JSP, etc.), que permiten generar contenido dinámico según las peticiones del usuario.
- Uso intensivo de **JavaScript** y tecnologías como **AJAX**, que permiten actualizar partes de una página sin recargarla completamente.
- Surge el concepto de **usuario participativo**, capaz de generar contenido (comentarios, publicaciones, perfiles, etc.).

Web 3.0 – Web semántica y aplicaciones inteligentes (actualidad)

- Apuesta por la **estructura semántica de los datos**, la interoperabilidad entre plataformas y la automatización de procesos.
- Uso de **APIs REST**, **ontologías**, **metadatos** y tecnologías de inteligencia artificial.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- Aplicaciones más complejas que funcionan como software de escritorio: editores de texto, gestores de proyectos, plataformas de administración electrónica.
- Aparición de conceptos como **Web progresiva** (PWA), **aplicaciones móviles híbridas** y uso de tecnologías emergentes como **WebAssembly**.

1.2 ARQUITECTURA CLIENTE-SERVIDOR

Las aplicaciones web se apoyan en una **arquitectura cliente-servidor**, modelo fundamental de funcionamiento de Internet. En este modelo, el trabajo se reparte entre dos componentes diferenciados:

Cliente (front-end)

- Es el navegador web que actúa como interfaz entre el usuario y la aplicación.
- Solicita recursos al servidor mediante el protocolo HTTP/HTTPS.
- Interpreta y ejecuta el contenido recibido (HTML, CSS, JavaScript) para representar la interfaz y permitir la interacción.
- Permite procesar parte de la lógica en el lado del cliente, mejorando la experiencia del usuario (por ejemplo, validaciones de formularios, dinámicas visuales, etc.).

Servidor (back-end)

- Se encarga de procesar las peticiones del cliente.
- Implementa la lógica de negocio de la aplicación y accede a bases de datos o servicios externos.
- Genera respuestas, que pueden consistir en páginas HTML completas o datos estructurados (JSON, XML) para que el cliente los procese.
- Utiliza lenguajes como PHP, Java, C#, Python, Node.js, entre otros.

Modelo de ejecución típico de una aplicación web

1. El usuario accede a una URL desde su navegador (cliente).
2. El navegador realiza una petición HTTP al servidor web correspondiente.
3. El servidor recibe la petición, ejecuta el código necesario (por ejemplo, consulta una base de datos) y genera una respuesta.
4. La respuesta (HTML, JSON, XML, etc.) se envía al navegador, que la interpreta y presenta al usuario.
5. El usuario puede interactuar con la aplicación mediante formularios, botones, enlaces, etc., generando nuevas peticiones al servidor o acciones locales mediante JavaScript.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Este modelo puede complementarse con arquitecturas avanzadas como:

- **Modelo de múltiples capas (multitier)**: separando presentación, lógica de negocio y acceso a datos.
- **Aplicaciones de una sola página (SPA)**: donde toda la lógica está en el cliente y el servidor solo proporciona datos.
- **Aplicaciones progresivas (PWA)**: que permiten el uso sin conexión, notificaciones y otras capacidades propias de las aplicaciones móviles nativas.

2. DESARROLLO WEB FRONT-END Y EN SERVIDOR

El desarrollo web se basa en la interacción entre dos partes fundamentales: el **front-end,** que representa la interfaz con la que el usuario interactúa, y el **back-end,** que gestiona la lógica de negocio, el procesamiento de datos y la comunicación con bases de datos.

El **front-end** se encarga de la experiencia de usuario (*UX/UI*), mientras que el **back-end** se ocupa de la lógica interna que permite que la aplicación funcione correctamente.

Característica	Front-end	Back-end
Definición	Parte visible de la aplicación con la que interactúa el usuario	Lógica de negocio, procesamiento de datos y comunicación con servidores
Lenguajes principales	HTML, CSS, JavaScript	PHP, Python, Java, C#, Node.js
Responsabilidad	Diseño de la interfaz, interacción con el usuario, animaciones, estructura visual	Procesamiento de datos, autenticación, acceso a bases de datos, seguridad
Ejecutado en	Navegador del usuario	Servidor web
Ejemplos de frameworks	React, Angular, Vue.js	Django, Express.js, Spring Boot, Laravel

2.1 TECNOLOGÍAS Y FRAMEWORKS FRONT-END

2.1.1 Lenguajes principales

1. **HTML (HyperText Markup Language)**
   - Lenguaje de marcado que define la estructura de la web.
2. **CSS (Cascading Style Sheets)**
   - Se usa para aplicar estilos y mejorar la apariencia de las páginas web.
   - Incluye técnicas avanzadas como ***Flexbox***, ***Grid*** y animaciones.
3. **JavaScript (JS)**
   - Lenguaje de programación que añade interactividad y dinamismo a las páginas.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- Permite gestionar eventos, manipular el DOM y realizar peticiones asincrónicas (*AJAX*).

2.1.2 Frameworks y bibliotecas más populares

Para facilitar el desarrollo de interfaces modernas, existen diversos **frameworks y bibliotecas** que optimizan el uso de JavaScript:

Nombre	Descripción	Ventajas principales
React.js	Biblioteca creada por Meta (Facebook) basada en **componentes reutilizables.**	Usa **Virtual DOM** para alto rendimiento y reactividad.
Angular	Framework completo desarrollado por Google, basado en **TypeScript**.	Arquitectura robusta, modular, CLI, **ideal para proyectos grandes.**
Vue.js	Framework progresivo, ligero y de **aprendizaje sencillo.**	Flexible, fácil de integrar, excelente para **proyectos pequeños y medianos.**

2.1.3 Herramientas complementarias.

Estas tecnologías permiten desarrollar interfaces atractivas, adaptables y optimizadas para cualquier dispositivo (*responsive design*).

- **Bootstrap**: framework CSS para diseño responsivo.
- **Tailwind CSS**: sistema de clases CSS altamente personalizable.
- **Webpack/Vite**: herramientas de construcción y optimización de código.

2.2 TECNOLOGÍAS Y FRAMEWORKS DE DESARROLLO EN SERVIDOR

1. **PHP**
   - Lenguaje tradicional para desarrollo web dinámico.
   - Compatible con servidores Apache y bases de datos MySQL.
   - Frameworks populares: **Laravel, Symfony**.
2. **Java**
   - Lenguaje robusto y escalable.
   - Utilizado en aplicaciones empresariales con **Spring Boot** y **Jakarta EE**.
4. **Python**
   - Popular en desarrollo web, inteligencia artificial y automatización.
   - Frameworks populares: **Django, Flask**.
4. **JavaScript (Node.js)**
   - Ejecuta JavaScript en el servidor con el motor **V8**.
   - Ideal para aplicaciones en tiempo real y microservicios.
   - Frameworks: **Express.js, NestJS**.
5. **C# (.NET Core)**
   - Utilizado en aplicaciones empresariales con **ASP.NET**.
   - Ofrece integración con servicios en la nube de Microsoft.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2.2.1 Arquitecturas y paradigmas

- **MVC (Modelo-Vista-Controlador)**: estructura común en frameworks como Django, Laravel y ASP.NET.
- **Microservicios**: modelo en el que cada funcionalidad de una aplicación es un servicio independiente (Docker, Kubernetes).
- **API REST y GraphQL**: métodos para la comunicación entre front-end y back-end.

2.2.2 Bases de datos y almacenamiento

Las aplicaciones web suelen depender de bases de datos para almacenar y recuperar información:

- **Relacionales (SQL)**: MySQL, PostgreSQL, SQL Server.
- **NoSQL**: MongoDB, Firebase, Redis.

3. DESARROLLO WEB MULTIPLATAFORMA Y MULTIDISPOSITIVO

En la actualidad, los usuarios acceden a aplicaciones web desde una amplia variedad de dispositivos: ordenadores de escritorio, portátiles, tablets, smartphones e incluso relojes inteligentes o televisores. Este escenario plantea la necesidad de desarrollar aplicaciones que se adapten de forma óptima a diferentes tamaños de pantalla, resoluciones y capacidades de hardware.

Para lograrlo, el desarrollo web ha evolucionado hacia **enfoques multiplataforma y multidispositivo**, con tecnologías como **Responsive Web Design (RWD)**, **Progressive Web Apps (PWAs)** y **frameworks híbridos**.

- **Diseño responsivo (Responsive Web Design - RWD)**: Adaptación del contenido a distintos tamaños de pantalla.
- **Frameworks multiplataforma**: Desarrollo de aplicaciones con una única base de código para diferentes dispositivos.
- **Progressive Web Apps (PWAs)**: Aplicaciones web con funcionalidades avanzadas similares a las apps nativas.

3.1 RESPONSIVE WEB DESIGN (DISEÑO WEB ADAPTATIVO)

El **Responsive Web Design (RWD)** es una técnica de desarrollo web que permite que un sitio se **adapte automáticamente** al tamaño y resolución del dispositivo en el que se visualiza, sin necesidad de desarrollar múltiples versiones de la misma web.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Principios del diseño responsivo

- **Mobile-first design:** Se diseña la aplicación primero para móviles y luego se expande a pantallas más grandes.
- **Uso de media queries en CSS**: Permiten aplicar estilos específicos según el tamaño de la pantalla. Se definen en CSS mediante la regla @media.
- **Diseño flexible con CSS Grid y Flexbox**. Evita el uso de medidas fijas y en su lugar usa porcentajes `(%)`, `vh` (altura del viewport) o `vw` (ancho del viewport).
- **Imágenes y contenido adaptable**. Se usa `max-width:100%` en imágenes para que se ajusten sin desbordar el contenedor.
- **Tipografía adaptable (REM y EM).** Se recomienda usar `rem` o `em` en lugar de `px` para escalar el tamaño de texto según la configuración del usuario.

3.2 PROGRESSIVE WEB APPS (PWAS)

Las **Progressive Web Apps (PWAs)** son aplicaciones web que ofrecen una experiencia similar a las aplicaciones móviles nativas, combinando lo mejor del desarrollo web con características avanzadas como uso sin conexión, notificaciones push y acceso al hardware del dispositivo.

Características principales de una PWA

- **Funcionan sin conexión** gracias a los **Service Workers**, que almacenan datos en caché.
- **Son instalables** en el dispositivo, como una app nativa.
- **Se ejecutan en pantalla completa**, sin barra de navegación del navegador.
- **Tienen rendimiento optimizado**, similar al de una aplicación móvil.



© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Componentes clave de una PWA

1. **Service Worker**: Un script en JavaScript que gestiona la caché y permite el funcionamiento sin conexión.
2. **Manifiesto Web App (manifest.json)**: Archivo que define el nombre de la PWA, iconos y apariencia en el dispositivo.
3. **HTTPS obligatorio**: Las PWAs requieren conexión segura para garantizar la integridad y seguridad de los datos.

Las PWAs ofrecen una **alternativa más ligera y flexible** a las aplicaciones nativas, con menor coste de desarrollo y mantenimiento.

3.3 FRAMEWORKS Y HERRAMIENTAS PARA EL DESARROLLO MULTIPLATAFORMA

Los **frameworks híbridos** permiten desarrollar **una única base de código** que puede ejecutarse tanto en **navegadores web como en dispositivos móviles (Android/iOS)**, evitando la necesidad de crear aplicaciones separadas para cada plataforma.

Estos frameworks permiten desarrollar aplicaciones con **una única base de código**, reduciendo el tiempo y costo de desarrollo.

Ionic

- Basado en HTML, CSS y JavaScript.
- Se integra con Angular, React o Vue.
- Usa **WebView** para renderizar contenido dentro de una aplicación nativa.

React Native

- Desarrollado por Meta (Facebook). Permite desarrollar **apps nativas** con JavaScript y React.
- Usa **JavaScript y JSX**, permitiendo compartir código entre la web y dispositivos móviles.
- **Compila a código nativo**, ofreciendo mayor rendimiento que los frameworks basados en WebView.
- Es utilizado por aplicaciones como Instagram, Facebook y Airbnb.

Flutter

- Desarrollado por Google
- Usa el lenguaje **Dart**.
- Es compatible con Android, iOS, web y aplicaciones de escritorio.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Xamarin

- Basado en .NET y C#, permite desarrollar aplicaciones nativas para Android, iOS y Windows.
- Ofrece integración completa con herramientas de Microsoft.

Tecnología	Uso principal	Rendimiento	Lenguaje	Plataforma
Responsive Web Design	Web adaptable	Depende del navegador	HTML, CSS, JavaScript	Web
PWA	Web con funciones avanzadas	Alto (con Service Workers)	HTML, CSS, JavaScript	Web, móvil
Ionic	Aplicaciones híbridas	Medio (usa WebView)	JavaScript, Angular, React	Android, iOS, Web
React Native	Apps casi nativas	Alto (compila a nativo)	JavaScript, JSX	Android, iOS
Flutter	Aplicaciones con alto rendimiento	Muy alto (motor gráfico propio)	Dart	Android, iOS, Web
Xamarin		Alto (compila a nativo)	C#	Android, iOS, Windows

3.4 ANDROID Y SUS VERSIONES

Los principales **frameworks híbridos y multiplataforma** (Ionic, React Native, Flutter, Xamarin) permiten desarrollar aplicaciones que pueden ejecutarse tanto en **Android como en iOS** sin necesidad de escribir código nativo para cada plataforma. Comprender Android y sus versiones es clave para optimizar aplicaciones en estos entornos.

Android es el sistema operativo móvil más utilizado en el mundo, desarrollado por Google y basado en el núcleo Linux. Fue diseñado para dispositivos móviles como teléfonos inteligentes, tabletas, relojes inteligentes y televisores.

Desde su lanzamiento en 2008, Android ha evolucionado con **mejoras en seguridad, rendimiento y funcionalidades**, introduciendo versiones numeradas junto con nombres de postres hasta Android 9 Pie. A partir de Android 10, Google eliminó los nombres de postres y adoptó una numeración más simple.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



Versión	Nombre	Año de lanzamiento	Características clave
Android 1.0	Apple Pie	2008	Primera versión comercial. Integración con Gmail, YouTube, y navegador web.
Android 1.1	Banana Bread	2009	
Android 1.5	Cupcake		
Android 1.6	Donut		
Android 2.2	Froyo	2010	Soporte para Flash y mejoras en rendimiento.
Android 4.0	Ice Cream Sandwich	2011	Nueva interfaz Holo, multitarea mejorada.
Android 4.1-4.3.1	Jelly Bean		
Android 4.4	KitKat		
Android 5.0	Lollipop	2014	Introducción de Material Design, soporte para 64 bits.
Android 6.0	Marshmallow	2015	Permisos de aplicaciones, Doze para ahorro de batería.
Android 8.0	Oreo	2017	Modo picture-in-picture, optimización en rendimiento.
Android 9.0	Pie	2018	
Android 10	-	2019	Eliminación de nombres de postres, modo oscuro, navegación por gestos.
Android 12	-	2021	Material You, seguridad mejorada, mejoras en rendimiento.
Android 14	-	2023	Mayor personalización, optimización de batería, mejoras en privacidad.

4. HTML, XML Y SUS DERIVACIONES

Los lenguajes de marcado son fundamentales para la creación, estructuración e intercambio de información en la web. Entre ellos destacan **HTML**, usado para estructurar contenidos visibles por el usuario, y **XML**, orientado a representar e intercambiar datos de forma estructurada.

4.1 HTML

HTML (HyperText Markup Language) es el lenguaje estándar utilizado para estructurar contenido en la web. Permite definir elementos como títulos, párrafos, tablas, formularios, imágenes, enlaces, entre otros, que forman la base visual de cualquier sitio web. HTML no es un lenguaje de programación, sino un **lenguaje de marcado**.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

1. Texto y contenido básico

Elemento	Uso
<h1> … <h6>	Títulos de diferentes niveles.
<p>	Párrafo.
 	Salto de línea.
<hr>	Línea horizontal (separador).
/ <i>	Texto en cursiva (semántico / visual).
	Contenedor genérico en bloque.

2. Enlaces e imágenes

Elemento	Uso
<a href="">	Enlace (hipervínculo).
<img>	Imagen. Atributos: src, alt, width, height.

3. Listas

Elemento	Uso
	Lista no ordenada.
	Lista ordenada.
	Elemento de lista.
<dl>	Lista de definiciones.
<dt>	Término a definir.



Recuerda: Cuando se vaya a crear un nuevo listado se utiliza ul, y los elementos se identifican con li.

4. Tablas

Elemento	Uso
<table>	Tabla.
<tr>	Fila de tabla.
<td>	Celda de datos.
<th>	Celda de encabezado.
<thead>, <tbody>, <tfoot>	Secciones de tabla.
<caption>	Título de la tabla.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

5. Formularios e interacción

Elemento	Uso
<form>	Formulario.
<imput>	Campo de entrada (texto, botón, checkbox, etc.).
<label>	Etiqueta para campos del formulario.
<textarea>	Campo de texto multilínea.
<select>	Lista desplegable.
<option>	Opción dentro de un <select>.
<button>	Botón (enviar, resetear, personalizado).
<fieldset>	Agrupación de controles.
<legend>	Título del grupo de campos.



Recuerda: La etiqueta <laberl>

- Se utiliza para **asociar una etiqueta textual** con un campo de formulario.
- El atributo for debe coincidir con el id del campo asociado. Enlaza el texto del <label> con un elemento de entrada por su id.

La etiqueta <input> se usa para **crear campos de entrada** en formularios HTML. Es una **etiqueta vacía**, lo que significa que **no necesita etiqueta de cierre**.

Principales tipos de input (type="...")

Tipo	Descripción
text	Campo de texto de una línea
password	Campo de texto con caracteres ocultos
email	Valida que el contenido tenga formato de email
number	Campo numérico (con flechas para subir/bajar)
date	Selector de fecha
checkbox	Casilla de verificación
radio	Botón de opción única (en grupo)
file	Permite subir archivos
submit	Botón para enviar el formulario
reset	Botón para reiniciar los campos del formulario
button	Botón sin función predefinida (se usa con JS)
range	Selector deslizante (de valores numéricos)

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Tipo	Descripción
color	Selector de color
search	Campo para búsquedas (con estilo específico)
tel	Para números de teléfono (con teclado adaptado en móviles)
url	Valida que el contenido sea una URL
datetime-local	Fecha y hora sin zona horaria

Atributos útiles de <input>

Atributo	Descripción
name	Nombre del campo (clave en el envío del formulario).
value	Valor inicial o por defecto.
placeholder	Texto guía que aparece dentro del campo.
required	Hace obligatorio rellenar el campo antes de enviar.
readonly	El campo se muestra pero no se puede editar.
disabled	El campo está deshabilitado (no se puede usar).
min / max	Establece el valor máximo y mínimo permitido
step	Incremento de valor permitido (number, range).
checked	Marca un checkbox o radio por defecto.
multiple	Permite múltiples selecciones (file, email).
pattern	Expresión regular para validar formato.
maxlength	Limita el número de caracteres máximos que se pueden introducir
size	Define el ancho visible de caracteres, NO EL NÚMERO MÁXIMO DE CARACTERES
autofocus	El campo gana el foco automáticamente al cargar la web.



Habitualmente puede aparecer un fragmento de código y hay que reconocer algunas de las funciones de los atributos principales, como maxlength, size o required.

Atributos de <form>

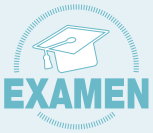
Atributo	Descripción
action	URL de destino a la que se envían los datos del formulario.
method	Verbo HTTP usado para el envío, comúnmente GET o POST.

Atributos globales id y class: pueden usarse en cualquier etiqueta HTML. La diferencia es que **id es único en el documento** (identifica un solo elemento), mientras que **class permite clasificar múltiples elementos** con un mismo nombre de clase.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

4.2 HTML5

HTML5 es la quinta y más reciente versión mayor del lenguaje HTML. Fue publicado como recomendación oficial por el **W3C en 2014** y ha supuesto una transformación significativa en el desarrollo web, al consolidar funcionalidades modernas que antes requerían plugins externos o múltiples tecnologías complementarias.



1. Nuevas Etiquetas semánticas de estructura: Permiten estructurar mejor el documento.

Etiqueta	Uso
<header>	Encabezado de una página o sección (no confundir con <head>).
<footer>	Pie de página o de sección.
<nav>	Bloque de navegación, enlaces principales.
<article>	Contenido independiente y autocontenido (como una noticia o entrada de blog).
<section>	Agrupación temática de contenido relacionado.
<aside>	Contenido lateral o complementario (como una barra lateral).
<main>	Contenido principal del documento (solo debe haber uno por página).
<figure>	Elemento gráfico independiente con posible título.
<figcaption>	Título o pie de imagen dentro de un <figure>.
<mark>	Resalta texto como si estuviera subrayado o marcado.
<time>	Representa una fecha u hora (útil para eventos, publicaciones, etc.).
<summary>	Título visible de un bloque desplegable (<details>).
<details>	Permite ocultar/mostrar contenido al hacer clic, sin JavaScript.

2. Tablas: En HTML5, las tablas siguen utilizando la misma estructura básica que en versiones anteriores de HTML. L**os cambios se centran en las etiquetas semánticas.**

Etiqueta	Uso
<table>	Contenedor principal de la tabla.
<caption>	Título descriptivo de la tabla (opcional).
<thead>	Encabezado de la tabla (una o varias filas).
<tbody>	Cuerpo principal de los datos.
<tfoot>	Pie de la tabla (sumas, notas, etc.).
<tr>	Table row – define una fila.
<th>	Table header – celda de encabezado (negrita y centrado por defecto).
<td>	Table data – celda de datos normales.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Otros atributos útiles

Atributo	Se aplica a	Descripción
colspan	<td>, <th>	Hace que una celda ocupe varias columnas.
rowspan	<td>, <th>	Hace que una celda ocupe varias filas.
scope	<th>	Mejora la accesibilidad indicando si el <th> es para fila o columna (row, col).



HTML5 incorpora nuevas etiquetas para la personalizado de las tablas

3. Soporte nativo para multimedia: HTML5 introduce soporte nativo para audio y vídeo sin plugins externos.

Etiqueta	Uso
<audio>	Incrustar audio. Permite controles de reproducción.
<video>	Incrustar vídeo. Soporta subtítulos, controles, etc.
<track>	Subtítulos, descripciones o traducciones para vídeos o audios.

Ejemplo para vídeo:

```
<video controls width="640" height="360">
  <source src="video.mp4" type="video/mp4">
  <source src="video.webm" type="video/webm">
  <source src="video.ogg" type="video/ogg">
  Tu navegador no soporta la reproducción de video HTML5.
</video>
```

- El navegador **lee los <source> de arriba hacia abajo**.
- Reproduce el **primer formato que soporte**.
- Si **ningún formato es compatible**, muestra el **mensaje de texto alternativo**.
- El atributo controls añade el **reproductor con botones**.

HTML5 introduce la etiqueta <audio> para permitir la **reproducción de archivos de sonido** directamente en el navegador, sin necesidad de plugins como Flash. Es compatible con formatos como .mp3, .ogg o .wav, según el navegador.

```
<audio src="audio.mp3" controls></audio>
```

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

O bien, con múltiples fuentes para mayor compatibilidad:

```
<audio controls>
  <source src="audio.mp3" type="audio/mpeg">
  <source src="audio.ogg" type="audio/ogg">
  Tu navegador no soporta el elemento de audio.
</audio>
```

Atributo	Descripción
controls	Muestra los controles de reproducción (play, pausa, volumen, etc.).
autoplay	Reproduce el audio automáticamente al cargar la página. (Puede estar bloqueado por algunos navegadores por razones de accesibilidad)
loop	El audio se reproduce en bucle.
muted	El audio se carga silenciado por defecto.
preload	Indica si el navegador debe cargar el audio al cargar la página. Valores: none, metadata, auto.

4. Formularios mejorados: HTML5 añade nuevos tipos de input y atributos que permiten validación y comportamiento mejorado sin JavaScript. **Nuevos tipos de <input type="…">**: email, url, tel, number, range, date, color, search, date-time-local

Etiqueta	Uso principal	Atributos clave
<progress>	Muestra una barra de progreso de una tarea	value, max
<meter>	Representa un valor dentro de un rango conocido	min, max, value, low, high, optimum
<output>	Muestra el resultado de un cálculo (JS o formulario)	for, name, id
<input type="range">	Control deslizante que permite seleccionar un valor	min, max, step, value
<input type="number">	Campo numérico con flechas para subir/bajar	min, max, step, value
<datalist>	Define una lista de sugerencias para un <input>	id
<color>	Permite mostrar un selector gráfico de color (paleta o rueda de color) nativo del navegador.	value
<range>	Proporciona un control deslizante para seleccionar un valor numérico dentro de un rango.	value (valor inicial), min y max (para los límites)
<date>	Muestra un calendario emergente que permite al usuario seleccionar una fecha fácilmente.	Valor en formato YYYY-MM-DD

4.3 CSS (CASCADING STYLE SHEETS)

Es un lenguaje que permite **definir la presentación visual** de los documentos HTML. Gracias a CSS, es posible aplicar estilos como colores, tamaños, márgenes, fuentes o posiciones a los elementos de una página web, manteniendo una **separación entre contenido y presentación**, lo que mejora la mantenibilidad y la reutilización del código. Una regla CSS se compone de tres partes:

```
selector {
```


© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

```
    propiedad: valor;
}
```

- **Selector**: indica a qué elementos se aplica la regla (en este caso, los <p>).
- **Propiedad**: define el aspecto a modificar (color, font-size, etc.).
- **Valor**: asigna un valor a la propiedad.

4.3.1. Orden de los selectores en CSS

La **especificidad** es el mecanismo por el cual el navegador decide **qué regla aplicar** cuando hay varias que afectan al mismo elemento. Se calcula con una fórmula basada en el tipo de selector:

```
(ID, Clases/Pseudo-clases, Etiquetas/Pseudo-elementos)
```

Selector	Especificidad
Selector de etiqueta	0-0-1
Clase, pseudo-clase	0-1-0
ID	1-0-0
Estilo en línea	1-0-0-0

Supongamos que queremos aplicar un estilo específico a los **campos obligatorios** de un formulario. Para ello, podemos utilizar el siguiente selector:

```
input:required {
  border: 2px solid red;
}
```

Este selector selecciona **todos los elementos <input> que tengan el atributo required**, es decir, los que son obligatorios en un formulario.

¿Cuál es su especificidad? El selector `input:required` combina:

- `input` → selector de tipo (etiqueta) → **0-0-1**
- `:required` → pseudo-clase → **0-1-0**

Total especificidad: 0-1-1

Orden en que aparecen en el CSS

Cuando dos reglas tienen la **misma especificidad**, gana la que esté **más abajo en la hoja de estilos** (la más reciente).

```
p {
  color: blue;
}
p {
  color: red; /* Este se aplica */
```

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

```
}
```

Selectores combinados aumentan especificidad. Cuanto más específico sea el selector, mayor prioridad tendrá.

```
p { ... }              → 0-0-1
#contenido p { ... } → 1-0-1
```

El comodín * tiene la menor prioridad

```
* {

  margin: 0;

}
```

El uso de !important. Forzará la aplicación de una regla, incluso si otra tiene más especificidad.

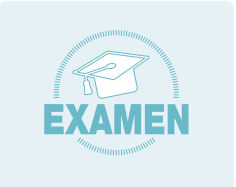
- No se recomienda abusar de `!important`, salvo en casos muy controlados.

```
p {

  color: red !important;

}
```

Tipos de selectores:



Nombre	Símbolo	Significado	Ejemplo

Descendiente	(Espacio)	Selecciona todos los elementos que están dentro de otro.	div p {
			color: blue;
			}

Hijo directo	>	Selecciona solo los elementos que son hijos directos del padre.
		En el ejemplo, aplica el estilo a los que están directamente dentro de un , pero no a los que están anidados más profundamente
			ul > li {
			font-weight: bold;
			}

Hermano adyacente	+	Selecciona un elemento que es el siguiente hermano inmediato del anterior.
		En el ejemplo, aplica el estilo al primer <p> que viene justo después de un <h2>.
			h2 + p {
			color: green;
			}

Hermanos generales	~	Selecciona todos los hermanos siguientes que comparten el mismo padre.
		En el ejemplo, aplica el estilo a todos los <p> posteriores a un <h2> dentro del mismo contenedor.
			h2 ~ p {
			font-style: italic;
			}

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Símbolo	Se usa para...	Ejemplo	Notas

#	Seleccionar por **ID**	#miElemento {
		color: red;
		}
		
		Texto en rojo
		
			Solo debería existir un único id por documento. No se repite el valor del id

.	Seleccionar por **clase**	.alerta {
		color: orange;
		}
		
		¡Atención!
		
			Reutilizable

4.3.2. Estilo en CSS:

Propiedades para tablas

Propiedad	Descripción
border	Añade borde a la tabla, celdas o filas.
border-collapse	Une los bordes de celdas adyacentes. (collapse o separate)
width	Ancho de la tabla. Puede ser en % o px.
text-align	Alineación horizontal del texto en celdas (left, center, right).
vertical-align	Alineación vertical (top, middle, bottom).
padding	Espacio interior en celdas.
background-color	Fondo de tabla, fila o celda.
color	Color del texto.
font-weight	Grosor del texto (normal, bold, etc.).

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Propiedades para transformaciones de texto

Propiedad	Descripción
Valor	Efecto en el texto
uppercase	Convierte todo el texto a **MAYÚSCULAS**.
lowercase	Convierte todo el texto a **minúsculas**.
capitalize	Convierte la primera letra de cada palabra a **mayúscula**.
none	Sin transformación (comportamiento por defecto).



Un preprocesador CSS es una herramienta que permite escribir CSS con funcionalidades avanzadas (variables, anidamiento, funciones o mixins, operaciones y reutilización de código), y a continuación, ese código se compila a CSS estándar que entiende el navegador. Algunos de los más conocidos son Sass y Less.

4.4 GRID LAYOUT EN CSS

CSS Grid Layout es un sistema de diseño bidimensional introducido en CSS3 que permite crear **diseños en filas y columnas** con gran precisión y flexibilidad. A diferencia de Flexbox (que trabaja mejor en una dimensión: horizontal o vertical), **Grid está pensado para gestionar ambas a la vez**, lo que lo hace ideal para estructuras complejas como maquetación de páginas, paneles o rejillas de contenido.

Para que un contenedor funcione como una cuadrícula (*grid container*), se le aplica:

```
.contenedor {
  display: grid;
}
```

Definir filas y columnas.

- `grid-template-columns:` define las columnas.
- `grid-template-rows:` define las filas.

Ejemplo, se crea na cuadrícula con **3 columnas** (una fija y dos proporcionales) y **2 filas automáticas**.

```
.contenedor {
  display: grid;
  grid-template-columns: 200px 1fr 2fr;
  grid-template-rows: auto auto;
```

Supera. by thePower © Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

```
}
```

Unidades útiles en Grid

Unidad	Significado
px, %	Valores fijos o relativos clásicos
fr	Fracción del espacio disponible
auto	Se adapta al contenido
minmax()	Define un mínimo y un máximo
repeat()	Repite valores fácilmente

Posicionar elementos en la cuadrícula

Se pueden colocar los elementos explícitamente usando:

- `grid-column-start, grid-column-end`
- `grid-row-start, grid-row-end`
- O abreviado: `grid-column, grid-row`

Espaciado y alineación

Propiedad	Función
gap	Espacio entre filas y columnas.
row-gap	Espacio entre filas.
column-gap	Espacio entre columnas.
justify-items	Alineación horizontal de los ítems.
align-items	Alineación vertical de los ítems.
place-items	Abreviación para justificar y alinear.

4.5 FLEXBOX (FLEXIBLE BOX LAYOUT)

Flexbox es un modelo de diseño en CSS que permite **distribuir y alinear elementos de forma flexible** dentro de un contenedor. A diferencia de Grid, que es bidimensional, Flexbox trabaja en **una sola dimensión**: horizontal (**fila**) o vertical (**columna**).

Propiedades del contenedor

Propiedad	Descripción
display: flex	Activa Flexbox en el contenedor.
flex-direction	Define la dirección del eje principal.
justify-content	Alineación en el eje principal (horizontal por defecto).
align-items	Alineación en el eje cruzado (vertical por defecto).
align-content	Alineación de múltiples líneas (cuando hay flex-wrap).
flex-wrap	Permite que los elementos pasen a la siguiente línea si no caben.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Dirección del eje (eje principal y eje cruzado)

- **Eje principal**: la dirección en la que se colocan los ítems (row, column, etc.).
- **Eje cruzado**: perpendicular al eje principal.

Se controla con:

```
flex-direction: row;        /* horizontal (por defecto) */

flex-direction: column;     /* vertical */

flex-direction: row-reverse;

flex-direction: column-reverse;
```

Propiedades de los elementos flexibles (hijos)

Propiedad	Descripción
flex-grow	Cuánto puede crecer un ítem si hay espacio disponible.
flex-shrink	Cuánto puede reducirse un ítem si hay falta de espacio.
flex-basis	Tamaño inicial del ítem antes de repartir espacio.
flex	Atajo para grow shrink basis. Ej: flex: 1 0 auto;
align-self	Alineación individual en el eje cruzado (sobrescribe align-items).
order	Permite reordenar visualmente los ítems.

Alineaciones más comunes con justify-content

Valor	Alineación
flex-start	A la izquierda (inicio del eje)
flex-end	A la derecha (final del eje)
center	Centrado
space-between	Espaciado con extremos alineados
space-around	Espaciado uniforme alrededor
space-evenly	Espaciado equitativo entre todos

4.6 BOOTSTRAP: FRAMEWORK DE DISEÑO WEB RESPONSIVO

Bootstrap es un **framework front-end** de código abierto que permite crear **interfaces web modernas y adaptables** de forma rápida y sencilla.

- Ofrece un conjunto de **estilos CSS y componentes ya listos para usar**.
- Incluye un **sistema de rejilla (grid)** para diseño responsive.
- Integra componentes como botones, formularios, menús, tarjetas, alertas, modales, etc.
- Incluye también **JavaScript** para interactividad básica (desplegables, carruseles, pestañas...).

¿Qué incluye?

- Hojas de estilo CSS predefinidas.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- Clases utilitarias (.text-center, .bg-primary, .mt-3, etc.).
- Componentes listos: botones (.btn), formularios (.form-control), navbars, etc.
- Sistema **mobile-first**, es decir, está optimizado para dispositivos móviles.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

5. XML: CARACTERÍSTICAS Y USOS

XML (eXtensible Markup Language) es un lenguaje de marcado extensible creado por el W3C para definir documentos con una estructura jerárquica y legible tanto por humanos como por máquinas. A diferencia de HTML, **XML no tiene etiquetas predefinidas**, sino que permite al usuario definirlas según las necesidades de los datos a representar.

Características principales de XML

- **Extensible**: el usuario define sus propias etiquetas.
- **Auto-descriptivo**: los datos están acompañados por su significado.
- **Bien formado**: requiere una sintaxis estricta (etiquetas cerradas, anidadas correctamente).
- **Validadas**: se puede validar su estructura mediante DTDs o XML Schemas.
- **Independiente de plataforma y lenguaje de programación.**

Etiquetas utilizadas en XML

Etiqueta o estructura	Uso o significado
<?xml … ?>	Declaración XML. Debe aparecer al inicio del documento. Define versión y codificación.
<!-- comentario -->	Comentarios. No se procesan ni muestran.
<etiqueta> … </etiqueta>	Etiqueta de apertura y cierre. Elemento básico de XML.
<etiqueta/>	Etiqueta vacía (autocontenida). Equivalente a apertura y cierre sin contenido.
<!DOCTYPE …>	Declaración del tipo de documento. Opcional, usada con DTD.
<!ELEMENT …>	Declaración de un elemento en un DTD.
<!ATTLIST …>	Declaración de atributos para elementos en un DTD.
<raíz>	Elemento raíz obligatorio. Debe envolver todo el contenido del documento XML.
<xsd:schema>	Raíz de un esquema XML (XML Schema Definition – XSD).
<xsd:element>	Define un elemento dentro de un esquema.
<xsd:attribute>	Define un atributo dentro de un esquema.
<xsl:stylesheet>	Raíz de una hoja de estilo XSLT (para transformar XML).
<xsl:template>	Plantilla de transformación XSLT.
<xsl:value-of>	Extrae el valor de un nodo.
<xop:Include>	Etiqueta usada en XOP para incluir datos binarios externos.

- XML **no tiene etiquetas predefinidas** como HTML. El usuario **define sus propias etiquetas** según la estructura de datos deseada.
- Las etiquetas deben:
  - » Abrirse y cerrarse correctamente.
  - » Estar bien anidadas.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

» Usar comillas en los atributos.

» Ser **sensible a mayúsculas y minúsculas** (<dato> ≠ <Dato>).

Derivaciones de XML más comunes

Nombre	Siglas / Extensión	Descripción / Uso principal
XHTML	eXtensible HTML	Versión de HTML reescrita en XML. Más estricto y bien formado.
SVG	Scalable Vector Graphics	Lenguaje para gráficos vectoriales escalables (iconos, ilustraciones, gráficos).
MathML	Mathematical Markup Language	Representación estructurada de expresiones matemáticas.
XSL	eXtensible Stylesheet Language	Lenguaje para transformar y dar formato a documentos XML.
XSLT	XSL Transformations	Transformaciones de XML a otros formatos (HTML, texto, XML...).
XSD	XML Schema Definition	Definición estructural y validación de datos XML (más potente que DTD).
RSS	Really Simple Syndication	Distribución automática de contenido actualizado (noticias, blogs).
SOAP	Simple Object Access Protocol	Intercambio estructurado de mensajes en servicios web.
WSDL	Web Services Description Language	Describe servicios web y sus métodos.
XBRL	eXtensible Business Reporting Language	Reportes financieros y contables estructurados.
GML	Geography Markup Language	Representación de información geográfica.
SCORM	Sharable Content Object Reference Model	Contenidos educativos reutilizables en plataformas e-learning.

Lenguajes de consulta y navegación en XML: XPath y XQuery

Lenguaje	Descripción / Uso
XPath (XML Path Language)	Lenguaje diseñado para **navegar y seleccionar elementos o atributos** dentro de un documento XML.
XQuery	Lenguaje funcional diseñado para **consultar colecciones de datos XML** (el equivalente de SQL para XML).

Diferencias entre HTML y XHTML

XHTML (Extensible HyperText Markup Language) es una **reformulación de HTML como aplicación XML**, lo que significa que **hereda las reglas estrictas de sintaxis de XML**, a diferencia de HTML, que es más permisivo.

Característica	HTML	XHTML
Sensibilidad a mayúsculas/ minúsculas	No es sensible (puede usar DIV, div, etc.)	Sensible: todas las etiquetas deben ir en **minúscula**.
Cierre de etiquetas	Algunas etiquetas pueden quedar sin cerrar (, <img>)	Todas las etiquetas deben cerrarse explícitamente: , <img />.
Atributos	Los atributos **pueden o no** llevar comillas	Los valores de los atributos deben ir **entre comillas.**
Elementos vacíos	Se permiten etiquetas auto-contenidas sin cierre	Se deben cerrar correctamente o usar la forma **autocontenida** (<input />).
Documento raíz	No requiere una estructura XML completa	Es obligatorio incluir un **elemento raíz** (<html>) y estar bien formado.

- Es una versión de HTML **reescrita en XML**.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- Requiere una sintaxis más estricta: todas las etiquetas deben cerrarse y anidarse correctamente.
- Aunque fue impulsado como el sucesor de HTML, fue reemplazado en la práctica por **HTML5**, que volvió a un modelo más flexible.

XOP (XML-binary Optimized Packaging)

XOP (*XML-binary Optimized Packaging*) es una especificación del **W3C** que define un mecanismo para **incluir datos binarios (como imágenes, audio o archivos)** dentro de documentos XML de forma más eficiente.

¿Por qué se necesita XOP?

- **XML solo permite datos en texto plano**.
- Incluir datos binarios directamente (por ejemplo, en base64) **aumenta el tamaño del mensaje** y **consume más recursos**.
- XOP permite **referenciar los datos binarios externamente**, manteniendo la estructura XML intacta.

DTD (Document Type Definition)

Un **DTD (Document Type Definition)** define la **estructura y las reglas** que debe seguir un documento XML para considerarse **válido**. Con un DTD, se puede especificar:

- Qué elementos pueden aparecer en el documento.
- En qué orden deben aparecer.
- Qué atributos puede tener cada elemento.
- Si un elemento es opcional o repetible.

Bien formado y válido

- Un documento XML que **cumple la sintaxis básica** se dice que está **bien formado**.
- Si además cumple las **reglas definidas por un DTD**, se dice que está **válido**.

» **Reglas básicas para estar bien formado:**

- Debe tener **un único elemento raíz**.
- Todas las **etiquetas deben cerrarse correctamente**.
- Las etiquetas deben estar **bien anidadas**.
- Los **atributos deben ir entre comillas**.
- **Sensibilidad a mayúsculas/minúsculas**: <nombre> ≠ <Nombre>.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

6. LENGUAJES DE SCRIPT. JAVASCRIPT

6.1 CARACTERÍSTICAS PRINCIPALES DE JAVASCRIPT

JavaScript es un lenguaje de programación **interpretado**, **imperativo, dinámico**, **orientado a objetos** y **basado en prototipos.** Está centrado principalmente en el desarrollo web del lado del cliente (aunque también se usa en el servidor con Node.js).

Característica	Descripción
Lenguaje interpretado	No necesita compilación previa. El navegador interpreta el código directamente.
Basado en objetos. Está basado en prototipos	Utiliza objetos y prototipos para organizar datos y funciones.
Tipado dinámico	Las variables no necesitan declaración de tipo (var x = 5;, x = "texto"; es válido).
Débilmente tipado	Se pueden mezclar tipos sin errores inmediatos, aunque esto puede producir errores lógicos.
Orientado a eventos	Responde a acciones del usuario (clics, envíos, teclas, etc.).
Multiplataforma	Funciona en todos los navegadores modernos y sistemas operativos.
Lenguaje de alto nivel	Fácil de leer y escribir, cercano al lenguaje humano.
Compatibilidad con HTML/CSS	Se integra perfectamente en páginas web para modificar contenido, estructura y estilos.
Asincronía y concurrencia	Soporta operaciones asíncronas con callbacks, promises y async/await.



Como recordatorio, algunos de los operadores más utilizados son los aritméticos:

- Suma (+)
- Resta (-)
- Multiplicación (*)
- División (/)
- Módulo (%)
- Potencia (**)
- Incremento (++)
- Decremento (- -)

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

6.2 EVENTOS COMUNES EN HTML (CLASIFICADOS POR TIPO)

Un **evento** es una **acción que ocurre en la página web**, generalmente provocada por el usuario (aunque también puede ser automática), que puede ser **detectada y respondida mediante JavaScript**. Los eventos permiten que una página web sea **interactiva**: reaccionar a clics, movimientos del ratón, envío de formularios, escritura en campos, carga de la página, etc.

Puede estar presente de dos formas:

1. HTML + JavaScript en línea:

```
<button onclick="saludar()">Haz clic aquí</button>

<script>

  function saludar() {

    alert("¡Hola! Has hecho clic en el botón.");

  }

</script>
```

- El botón tiene un **evento `onclick`** (se dispara al hacer clic).
- Al activarse, llama a la función `saludar()`, que muestra un mensaje.

2. HTML + JavaScript con addEventListener (más recomendado):

```
<button id="miBoton">Púlsame</button>

<script>

  const boton = document.getElementById("miBoton");

  boton.addEventListener("click", function() {

    alert("¡Has pulsado el botón!");

  });

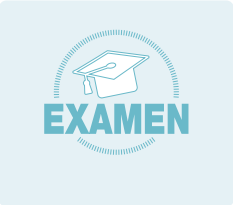
</script>
```

Eventos de ratón

Evento	Descripción
onclick	Se produce al hacer clic.
ondblclick	Doble clic.
onmouseover	El ratón entra en el área del elemento.
onmouseout	El ratón sale del área del elemento.
onmousedown	Se presiona un botón del ratón.
onmouseup	Se suelta el botón del ratón.
onmousemove	Movimiento del ratón dentro del elemento.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Eventos de teclado



Evento	Descripción
onkeydown	Se presiona una tecla.
onkeyup	Se suelta una tecla.
onkeypress	Se presiona y se mantiene una tecla (obsoleto, evitar en código nuevo).

Eventos de formulario

Evento	Descripción
onsubmit	Cuando se envía un formulario.
onreset	Cuando se reinicia un formulario.
onchange	Cuando cambia el valor de un campo (al perder el foco).
oninput	Cuando el usuario introduce datos (más inmediato que onchange).
onfocus	Cuando un elemento recibe el foco.
onblur	Cuando un elemento pierde el foco.

Eventos de ventana/documento



Evento	Descripción
onload	Cuando se ha cargado completamente la página o recurso.
onunload	Cuando el usuario abandona la página.
onresize	Cuando cambia el tamaño de la ventana.
onscroll	Cuando el usuario hace scroll en la página o un contenedor.

6.3 EXTRACCIÓN DE ELEMENTOS DEL DOM EN HTML CON JAVASCRIPT

El **DOM (Document Object Model)** es una **representación estructurada del documento HTML**, que convierte cada etiqueta en un **nodo o elemento** accesible y manipulable mediante JavaScript.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

JavaScript proporciona varios métodos para e**xtraer o seleccionar** elementos del DOM.

Métodos de selección del DOM en JavaScript

Método	Descripción	Tipo de resultado
getElementById()	Selecciona **un solo elemento** por su atributo id.	Elemento (HTMLElement)
getElementsByClassName()	Selecciona **todos los elementos con una clase específica**.	HTMLCollection
getElementsByTagName()	Selecciona **todos los elementos por etiqueta** (div, li, p, etc.).	HTMLCollection
querySelector()	Devuelve **el primer elemento** que coincida con un **selector CSS.**	Elemento (HTMLElement)
querySelectorAll()	Devuelve **todos los elementos** que coincidan con un **selector CSS.**	NodeList (iterable)

Propiedades de los nodos más comunes

Propiedad	Descripción	Ejemplo
firstChild	Devuelve el **primer hijo** de un nodo. Puede ser un nodo de texto o elemento.	element.firstChild
lastChild	Devuelve el **último hijo** del nodo.	element.lastChild
childNodes	Devuelve una lista (NodeList) de **todos los nodos hijos** (incluye textos).	element.childNodes
parentNode	Devuelve el **nodo padre**.	element.parentNode
nextSibling	Devuelve el **nodo siguiente** al mismo nivel.	element.nextSibling
previousSibling	Devuelve el **nodo anterior** al mismo nivel.	element.previousSibling
value	Devuelve o establece el **valor actual de un control de formulario** (como <input>, <select>, <textarea>).	document.getElementById("nombre").value

nodeValue: Se usa comúnmente con `firstChild` para acceder al contenido de texto de un elemento:

Propiedad	Descripción	Ejemplo
nodeValue	Devuelve o establece el **valor del nodo**, normalmente usado con nodos de texto.	element.firstChild.nodeValue

Alternativas

Método / Propiedad	Uso preferido actual
textContent	Devuelve o establece el **contenido de texto.**
innerHTML	Devuelve o establece el **HTML interno.**

6.4. JSON

JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos, basado en texto, fácil de leer y escribir por humanos y fácil de interpretar por máquinas. Es utilizado para:

- Enviar y recibir datos entre el cliente y el servidor.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- Almacenar configuraciones o estructuras de datos.

Estructura básica de JSON

```
{
  "nombre": "Lucía",
  "edad": 25,
  "activo": true,
  "intereses": ["lectura", "ciencia", "tecnología"]
}
```

Reglas básicas:

- Las **propiedades deben ir entre comillas dobles** ("clave").
- Los valores pueden ser: string, number, boolean, array, object o null.
- No se permiten funciones ni comentarios en JSON puro.

El **tipo MIME** oficial para JSON es **application/json** y su extensión de archivo es **.json**.

Conversión entre JSON y JavaScript

1. De objeto JavaScript a JSON (serializar) -> JSON.stringify()

```
const persona = {
  nombre: "Lucía",
  edad: 25
};
const jsonString = JSON.stringify(persona);
console.log(jsonString); // {"nombre":"Lucía","edad":25}
```

2. De JSON a objeto JavaScript (parsear) -> JSON.parse()

```
const datos = '{"nombre":"Lucía","edad":25}';
const obj = JSON.parse(datos);
console.log(obj.nombre); // Lucía
```

Método JS	Qué hace
JSON.stringify(obj)	Convierte un objeto JS en texto JSON
JSON.parse(texto)	Convierte un texto JSON en objeto JS
console.log()	Muestra datos en consola para depuración

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Errores comunes con JSON

Error	Descripción	Ejemplo incorrecto
Comillas simples	JSON solo permite **comillas dobles**	{ 'nombre': 'Ana' }
Comentarios	JSON **no permite comentarios**	// esto no es válido
Propiedades sin comillas	Las claves deben tener comillas	{ nombre: "Ana" }

6.5 FRAMEWORKS Y BIBLIOTECAS DE JAVASCRIPT

A medida que las aplicaciones web se han vuelto más complejas, han surgido numerosos **frameworks y bibliotecas de JavaScript** que facilitan el desarrollo, la organización del código y la creación de interfaces dinámicas y escalables.

Nombre	Descripción / Características
React	Desarrollada por Meta (Facebook), basada en componentes reutilizables y Virtual DOM.
Angular	Desarrollado por Google. Completo y basado en TypeScript. Incluye enrutamiento, servicios, etc.
Vue.js	Ligero y progresivo. Fácil de aprender. Ideal para escalar desde pequeños proyectos a grandes.
Svelte	Compila a JS puro (sin Virtual DOM). Muy rápido y con sintaxis clara.
Next.js	Extensión de React para renderizado del lado del servidor (SSR) y rutas automáticas.
Node.js	Permite usar JavaScript en el servidor. Basado en el motor V8 de Chrome.
Express.js	Framework minimalista para crear servidores y APIs REST.
jQuery	Simplifica la manipulación del DOM y AJAX (menos usado en proyectos modernos).
Meteor	Full-stack en tiempo real. Integra frontend y backend con base en Node.js.
Ember.js	Completo, estructurado y orientado a productividad.
Bootstrap	Framework CSS con componentes visuales, grid system y utilidades. Incluye algo de JS básico.
Jest	Framework de testing para JavaScript. Permite escribir pruebas unitarias y de integración.

- **React** y **Vue** se consideran más **flexibles** y requieren decidir qué librerías adicionales usar.
- **Angular** es un framework **"todo en uno"**, ideal para proyectos grandes y estructurados.
- **Node.js** + **Express.js** es la base más común para crear **backends con JavaScript**.
- **jQuery** fue muy popular, pero ha sido desplazado por frameworks modernos.

6.6 AJAX

AJAX (Asynchronous JavaScript and XML) se basa en un **modelo de comunicación asíncrona entre cliente y servidor**, lo que permite **actualizar contenido de una página web sin necesidad de recargarla por completo**.

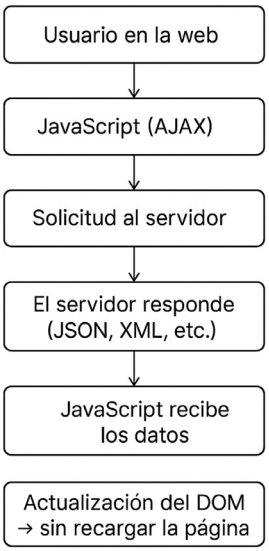
© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Modelo cliente-servidor asíncrono

En el modelo tradicional, cada vez que un usuario interactúa con un formulario o hace clic en un botón que requiere datos nuevos, el navegador **envía una solicitud al servidor y recarga toda la página** para mostrar el resultado. Con AJAX:

1. El usuario realiza una acción (por ejemplo, hace clic en un botón).
2. JavaScript, a través del objeto XMLHttpRequest o fetch(), **envía una solicitud al servidor en segundo plano**.
3. El servidor procesa la solicitud y **devuelve una respuesta (normalmente en JSON o XML)**.
4. JavaScript recibe la respuesta y **actualiza el contenido de la página de forma dinámica**, sin recargar.z

Flujo básico de una petición AJAX



Métodos para hacer peticiones AJAX

Método	Descripción
XMLHttpRequest	Método clásico para enviar y recibir datos de forma asíncrona. Requiere más código y manejo manual de estados.
fetch()	API moderna y recomendada. Basada en promesas. Sintaxis más limpia y legible.
Comparación general	XMLHttpRequest es el enfoque tradicional; fetch() es el estándar actual. Ambos permiten solicitudes asíncronas.

Uso de fetch(): método para AJAX

fetch() es una **API moderna de JavaScript** que permite realizar peticiones HTTP de forma sencilla y asíncrona. Sustituye a XMLHttpRequest con una **sintaxis más limpia**, basada en **promesas**, y se ha convertido en el **método estándar para AJAX** en el desarrollo web actual.

```
fetch(url)
  .then(respuesta => respuesta.json()) // o .text(), .blob(), etc.
  .then(datos => {
    // Usar los datos recibidos
    console.log(datos);
  })
  .catch(error => {
```

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

```
  console.error("Error en la petición:", error);
});
```

- **url**: dirección del recurso (archivo JSON, endpoint de API, etc.).
- **then()**: se encadena para procesar la respuesta.
- **catch()**: gestiona errores de red o peticiones fallidas.

Elementos principales en AJAX

Elemento	Descripción	Ejemplo / Valor típico
URL	Dirección del recurso al que se envía la petición.	"datos.php", "https://api.ejemplo.com/data"
Método HTTP	Tipo de operación: GET, POST, PUT, DELETE, etc.	"GET", "POST"
Cabeceras (headers)	Información adicional como el tipo de contenido.	{ "Content-Type": "application/json" }
Datos enviados (body)	Contenido enviado al servidor (en POST, PUT, etc.).	JSON.stringify({ nombre: "Ana" })
Tipo de respuesta esperada	Formato que devuelve el servidor: JSON, texto, XML...	response.json(), response.text()
Manejo de respuesta	Código para gestionar la respuesta exitosa.	.then(data => mostrar(data))
Manejo de errores	Código para capturar errores en la petición o conexión.	.catch(error => console.error(error))
Asincronía	Se realiza en segundo plano, sin bloquear la página.	Implicado automáticamente con fetch()
Actualización del DOM	Cambio visual basado en la respuesta recibida.	document.getElementById("salida").textContent = data.msg

Formatos de intercambio de datos en AJAX

Formato	Descripción	Ventajas
JSON	Formato de texto ligero basado en objetos JavaScript. Es **el más usado** actualmente.	√ Fácil de leer y parsear en JS (JSON.parse(), JSON.stringify()) √ Ligero y eficiente
XML	Formato estructurado y jerárquico basado en etiquetas. Originalmente usado con AJAX.	√ Estandarizado y extensible √ Puede validarse con DTD/XSD
Texto plano	Cadena de texto simple sin formato estructurado.	√ Muy ligero y fácil de transmitir

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

7. LENGUAJES DE SCRIPT. PHP

PHP (Hypertext Preprocessor) es un **lenguaje de programación del lado del servidor** diseñado específicamente para el desarrollo web. Permite crear páginas dinámicas que se generan en el servidor antes de enviarse al navegador del usuario. **Se ejecuta en el servidor, no en el navegador**.

```
<?php
  echo "Hola, mundo desde PHP";
?>
```

Característica	Descripción
Interpretado	No requiere compilación previa. El código se ejecuta directamente en el servidor.
Orientado a la web	Diseñado desde el principio para crear páginas dinámicas.
Multiplataforma	Funciona en Linux, Windows, macOS, etc.
Amplia comunidad y recursos	Muchas librerías, frameworks (Laravel, Symfony) y documentación disponible.
Integración con bases de datos	Soporta MySQL, PostgreSQL, SQLite, Oracle, entre otros.

PHP se puede incrustar directamente en un archivo .php, mezclando HTML y código del lado del servidor:

```
<!DOCTYPE html>
<html>
<body>
  <h1>Bienvenido</h1>
  <?php
    echo "Hola desde PHP";
  ?>
</body>
</html>
```

- El código PHP se escribe entre **<?php … ?>.**
- También puede usarse <?= … ?> como atajo de echo.
- Las etiquetas cortas **<? … ?>** (Short Open Tags), forma alternativa desaconsejada, dependen de la directiva **short_open_tag** del archivo **php.ini**, mientras que **<?php … ?>** está siempre garantizado. La etiqueta abreviada de impresión <?= … ?> está disponible siempre desde PHP 5.4, con independencia de esa directiva.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Comentarios en PHP

- **Una línea**: // o #
- **Varias líneas**: /* ... */

Variables

- Se definen con el símbolo $.
- No es necesario declarar el tipo de dato.

Constantes

- Se definen con `define()` o `const` (desde PHP 5.3).

Variables predefinidas (superglobales) en PHP

Variable	Descripción	Ejemplo de uso
$_GET	Almacena datos enviados mediante el método GET (en la URL).	$_GET["nombre"]
$_POST	Almacena datos enviados mediante el método POST (formularios).	$_POST["email"]
$_REQUEST	Contiene datos de $_GET, $_POST y $_COOKIE.	$_REQUEST["dato"]
$_SERVER	Información del servidor y del entorno de ejecución.	$_SERVER["HTTP_USER_AGENT"]
$_SESSION	Almacena datos de sesión del usuario (requiere session_start()).	$_SESSION["usuario"]
$_COOKIE	Accede a las cookies enviadas por el navegador.	$_COOKIE["idioma"]

Variable	Descripción	Ejemplo de uso
$_FILES	Gestiona archivos subidos a través de formularios.	$_FILES["archivo"]["name"]
$_ENV	Variables de entorno del sistema (depende del servidor y configuración).	$_ENV["PATH"]
$_GLOBALS	Permite acceder a todas las variables globales definidas en el script.	$_GLOBALS["x"]
$_SESSION	Persiste información entre páginas durante la sesión del usuario.	$_SESSION["carrito"][] = $producto;

- Estas variables están **disponibles en cualquier parte del script**: funciones, clases, archivos externos, etc.
- Todas son **arrays asociativos**.
- Suelen utilizarse para **formularios, autenticación, sesiones, subida de archivos y seguridad**.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Tipos de datos

Tipo	Ejemplo
String	"Hola"
Integer	42
Float	3.14
Boolean	true, false
Array	[1, 2, 3]
Object	Instancia de una clase
NULL	Sin valor

Operadores aritméticos

Operador	Significado	Ejemplo
+	Suma	$a + $b
-	Resta	$a - $b
*****	Multiplicación	$a * $b
/	División	$a / $b
%	Módulo	$a % $b

Operadores de asignación

Operador	Ejemplo	Equivalente a
=	$a = 5	
+=	$a += 2	$a = $a + 2
-=	$a -= 2	$a = $a - 2
***=**	$a *= 2	$a = $a * 2
/=	$a /= 2	$a = $a / 2

Operador de concatenación

- Se usa el punto (.) para **unir cadenas de texto**:

```
$nombre = "Ana";

$saludo = "Hola, " . $nombre;

echo $saludo; // Hola, Ana
```

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Estructuras de control en PHP. Condicionales en PHP

Estructura	Descripción
if	"Hola"
else	42
elseif	Ejecuta código si se cumple una condición (flujo secuencial de evaluación).
switch	Compara una misma variable contra varios valores posibles. Ideal cuando hay muchas condiciones fijas.

Estructuras de control: Bucles en PHP

Bucle	Descripción
for	Se usa cuando se **conoce el número de repeticiones**.
while	Ejecuta el bloque **mientras la condición sea verdadera**.
do...while	Ejecuta **al menos una vez**, incluso si la condición es falsa.
foreach	Recorre **cada elemento de un array**. Ideal para arrays indexados o asociativos.

Uso de break y continue

Instrucción	Uso principal
break	Sale **inmediatamente** del bucle o switch.
continue	**Salta al siguiente ciclo** sin ejecutar el resto del código actual.

Funciones comunes con cadenas (strings)



Función	Descripción	Ejemplo	Resultado
strlen()	Devuelve la **longitud** de una cadena	strlen("Hola")	4
substr()	Extrae una **subcadena**	substr("PHP7", 0, 3)	"PHP"
strtolower()	Convierte todo a **minúsculas**	strtolower("TEXTO")	"texto"
strtoupper()	Convierte todo a **mayúsculas**	strtoupper("php")	"PHP"
str_replace()	Reemplaza texto dentro de una cadena	str_replace("mundo", "PHP", "Hola mundo")	"Hola PHP"
trim()	Elimina **espacios** al inicio y final	trim(" hola ")	"hola"
strpos()	Busca la posición de una subcadena	strpos("Hola PHP", "PHP")	5
str_shuffle()	La función str_shuffle() en PHP se utiliza para mezclar aleatoriamente los caracteres de una cadena.	echo str_shuffle("abc123");	Salidas posibles: b1a2c3 3cab21 231abc

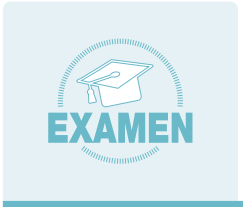
© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Formateo de fechas con date()

La función date() permite **dar formato a la fecha y hora actual** o de una marca de tiempo (timestamp).

```
date("formato", timestamp_opcional);
```

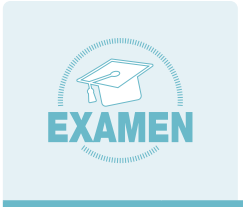
Funciones comunes con arrays



Función	Descripción	Resultado
count()	Devuelve el número de elementos del array.	int
array_push()	Añade uno o más elementos al final del array.	Modifica el array
array_pop()	Elimina y devuelve el **último** elemento del array.	Último elemento
array_shift()	Elimina y devuelve el **primer** elemento del array.	Primer elemento
array_unshift()	Añade uno o más elementos al **inicio** del array.	Modifica el array
in_array()	Comprueba si un valor existe en el array.	true / false
array_key_exists()	Verifica si una clave existe en un array asociativo.	true / false
array_keys()	Devuelve todas las claves del array.	Array de claves
array_values()	Devuelve todos los valores del array.	Array de valores
array_merge()	Combina dos o más arrays.	Nuevo array combinado
array_slice()	Extrae una porción del array.	Subarray con 2 elementos
array_reverse()	Devuelve el array en orden inverso.	Nuevo array invertido
sort()	Ordena un array **indexado** en orden ascendente.	Modifica el array
rsort()	Ordena un array en orden descendente.	Modifica el array
asort()	Ordena un array **asociativo por valores** manteniendo las claves.	Ordenado por valores
ksort()	Ordena un array asociativo **por claves**.	Ordenado por claves

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Framework en PHP



Framework	Descripción	Ventajas principales	Ideal para...
Laravel	El framework PHP más popular. Elegante, moderno y con enfoque en productividad.	Sintaxis clara, sistema de rutas potente, migraciones, Eloquent ORM, Blade.	Aplicaciones web modernas y complejas
Symfony	Framework robusto, reutilizable y modular. Base de Laravel.	Muy flexible, arquitectura sólida, componentes reutilizables.	Proyectos grandes y personalizables
CodeIgniter	Framework ligero y rápido. Fácil de instalar y usar.	Curva de aprendizaje baja, rendimiento alto, sin dependencias externas.	Aplicaciones pequeñas o medianas
Yii	Framework de alto rendimiento. "Yes, it is!"	Integrado con Gii (generador de código), caché, seguridad, RESTful API.	Aplicaciones empresariales seguras
CakePHP	Framework basado en convenciones. Inspirado en Ruby on Rails.	Rápido para empezar, CRUD automático, estructura MVC clara.	Prototipado rápido, desarrollo ágil
Phalcon	Framework muy rápido, implementado como extensión C de PHP.	Alto rendimiento, consumo reducido de recursos.	Aplicaciones que requieren velocidad
Slim	Microframework para APIs REST y servicios web.	Ligero, muy fácil de aprender, ideal para backends pequeños.	APIs o microservicios
Lumen	Microframework de Laravel, centrado en rendimiento.	Rápido, simple, mantiene compatibilidad con componentes de Laravel.	APIs y microservicios con Laravel
FuelPHP	Flexible y con soporte para HMVC.	Arquitectura modular, seguridad integrada, orientación a objetos.	Proyectos medianos y avanzados
Zend Framework / Laminas	Framework empresarial (actualmente Laminas Project).	Arquitectura modular, usado en entornos corporativos.	Aplicaciones empresariales críticas

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

8. LENGUAJE DE PROGRAMACIÓN PYTHON

Python es un lenguaje de programación de **alto nivel**, **interpretado**, de propósito general y con una **sintaxis clara y legible**. Fue creado por **Guido van Rossum** y su primera versión se publicó en 1991.

Python es un lenguaje de programación de **alto nivel**, **interpretado** y **de propósito general**, que se caracteriza por una sintaxis clara, legible y elegante. Fue diseñado para ser fácil de aprender y utilizar, lo que lo convierte en uno de los lenguajes más populares tanto para principiantes como para profesionales.

Es un lenguaje **multiparadigma**, lo que significa que permite programar de forma **imperativa**, **orientada a objetos** y **funcional**.

Característica	Descripción
Sencillo y legible	Su sintaxis se asemeja al lenguaje natural, lo que facilita su aprendizaje.
Interpretado	No requiere compilación previa; se ejecuta línea por línea.
Multiparadigma	Soporta programación imperativa, orientada a objetos y funcional.
Tipado dinámico	No es necesario declarar el tipo de las variables.
Multiplataforma	Funciona en Windows, Linux, macOS, etc.
Gran ecosistema	Miles de librerías y frameworks disponibles (Django, Flask, NumPy, etc.).

8.1 SINTAXIS Y DATOS

Sintaxis básica

- Python **no usa llaves {} ni punto y coma ;**.
- La estructura del código se define por **indentación obligatoria** (recomendado: 4 espacios).
- Las líneas que comienzan con # son **comentarios**.



En Python, no se usan llaves {}, corchetes [] ni paréntesis ().

Los bloques se identifican por sangría (indentación)

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Tipos de datos primitivos (básicos)

Tipo	Descripción	Ejemplos
int	Números enteros, positivos o negativos	5, -12, 0
float	Números decimales (con punto flotante)	3.14, -0.5, 1.0
str	Cadenas de texto, entre comillas simples o dobles. Pueden concatenarse o repetirse.	'hola', "hola""hi" * 3 → 'hihihi'
bool	Booleanos: valores lógicos usados en condiciones	True, False
None	Representa **ausencia de valor.** Similar a null en otros lenguajes.	None

Conversión de tipos

Conversión	Ejemplo	Resultado
int()	int("5")	5 (entero)
float()	float("3.14")	3.14 (decimal)
str()	str(42)	"42" (cadena)
bool()	bool(0) → False, bool(1) → True	Valor lógico

8.2 VARIABLES Y CONSTANTES

Las **variables** permiten almacenar valores para usarlos más adelante. Aunque en Python **no existen constantes verdaderas**, se pueden definir por **convención**.

Reglas de nombrado de variables

Para definir nombres válidos de variables en Python, hay que seguir estas reglas:

√ Permitido	Θ Prohibido
Letras minúsculas y mayúsculas (a–z, A–Z)	Nombres que **empiezan por un número** (1variable)
Números (0–9) **después del primer carácter**	**Espacios** en los nombres (nombre completo)
Guiones bajos (_)	**Símbolos especiales** (@, $, %, !, etc.)
Deben comenzar con **letra o** _ (nombre, _x)	Uso de **palabras reservadas** (if, while, class, etc.)

Asignación de valores

La asignación en Python se hace con el operador =. No es necesario declarar el tipo antes:

```
x = 10          # int

nombre = "Ana" # str

pi = 3.14       # float

activo = True   # bool
```


© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Constantes por convención

Python **no tiene constantes verdaderas** como en otros lenguajes (no se puede declarar const). Sin embargo, por **convención**, se escriben en **mayúsculas** y **no deben modificarse**. Ojo, técnicamente se puede cambiar, pero no se debe.

```
PI = 3.1416

MAX_USUARIOS = 100
```

Asignación múltiple

Python permite asignar **múltiples variables a la vez**:

```
a, b, c = 1, 2, 3

print(a, b, c)  # 1 2 3
```

También se pueden asignar **el mismo valor a varias variables**:

```
x = y = z = 0
```

8.3 OPERADORES EN PYTHON

Operadores aritméticos

Operador	Descripción	Ejemplo	Resultado
+	Suma	3 + 2	5
-	Resta	7 - 4	3
*	Multiplicación	3 * 2	6
/	División (float)	10 / 4	2.5
//	División entera	10 // 4	2
%	Módulo (resto)	10 % 3	1
**	Potencia	2 ** 3	8



Si utilizas un operador como * para multiplicar una cadena de texto, esta se repetirá tantas veces como se indica tras el operados.

Operadores de comparación

Devuelven **True** o **False** al comparar valores.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Operador	Descripción	Ejemplo	Resultado
==	Suma	3 + 2	5
Igual	5 == 5	True	3
!=	Distinto	5 != 3	True
>	Mayor que	7 > 4	True
<	Menor que	2 < 1	False
>=	Mayor o igual que	3 >= 3	True
<=	Menor o igual que	5 <= 6	True

Operadores lógicos

Operador	Descripción	Ejemplo	Resultado
and	Verdadero si ambas son True	True and False	False
or	Verdadero si alguna es True	True or False	True
not	Invierte el valor booleano	not True	False

Valores Truthy y Falsy

En Python, cualquier valor puede evaluarse en una condición (por ejemplo, en un if). Además de **False**, se evalúan como falsos: el número **0**, **None**, las **colecciones vacías** ([], {}, ()) y las **cadenas vacías** (""). Todo lo demás es **Truthy**.

Valor	Evaluación	Ejemplo
False, None, 0	**Falsy**	if 0: → no se ejecuta
[], {}, (), ""	**Falsy**	if []: → no se ejecuta
Cualquier otro valor (5, [0], "a"...)	**Truthy**	if [0]: → sí se ejecuta

Operadores de asignación

Operador	Ejemplo	Equivalente a
=	x = 5	Asigna valor
+=	x += 1	x = x + 1
-=	x -= 2	x = x - 2
*=	x *= 3	x = x * 3
/=	x /= 4	x = x / 4
//=	x //= 2	x = x // 2
%=	x %= 3	x = x % 3
**=	x **= 2	x = x ** 2

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Operadores de identidad y pertenencia

Tipo	Operadores	Descripción	Ejemplo	Resultado
Identidad	is, is not	Comprueba si dos variables apuntan al mismo objeto en memoria.	a = [1, 2] b = a a is b	True
Pertenencia	in, not in	Verifica si un elemento está o no está en una colección (lista, cadena, etc.).	letras = "python" " y" in letras "z" not in letras	True True

8.4 CONTROL DE FLUJO EN PYTHON

Estructura	Función principal
if / elif / else	Tomar decisiones según condiciones
for	Recorrer listas, rangos, cadenas, etc.
while	Repetir mientras una condición sea verdadera
break	Salir del bucle
continue	Saltar a la siguiente iteración
pass	No hacer nada (relleno)
List comprehension	Crear listas de forma concisa y eficiente

8.5 FUNCIONES EN PYTHON

Las **funciones** permiten agrupar instrucciones que se pueden reutilizar.

Definición con def

Se define una función con la palabra clave **def**, seguida del nombre y paréntesis:

```
def saludar():
    print("Hola, mundo")
```

Parámetros y retorno

Las funciones pueden recibir **parámetros** (entradas) y devolver un **resultado** con return.

```
def sumar(a, b):
    return a + b
resultado = sumar(3, 5)
print(resultado)  # 8
```

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

8.6 ESTRUCTURAS DE DATOS EN PYTHON

Listas

Colecciones **ordenadas y mutables**. Se pueden modificar, añadir y eliminar elementos.

```
frutas = ["manzana", "pera", "uva"]
```

Método	Descripción	Ejemplo
.append(x)	Añade un elemento al final	frutas.append("naranja")
.insert(i, x)	Inserta en una posición concreta	frutas.insert(1, "kiwi")
.remove(x)	Elimina un valor	frutas.remove("uva")
.pop(i)	Elimina y devuelve el valor en i	frutas.pop(0)
.sort()	Ordena la lista	frutas.sort()
.reverse()	Invierte el orden	frutas.reverse()

Acceso a elementos en listas

Las listas se acceden por **índice numérico**, sin comillas. Los **índices empiezan en 0**

```
numeros = [10, 20, 30]

print(numeros[0])  # → 10
```

Tuplas

Colecciones **ordenadas pero inmutables**. Usadas para datos que **no deben cambiar**.

```
coordenadas = (10, 20)
```

- No se pueden modificar (append, remove, etc. no existen).
- Se accede por índice: coordenadas[0] → 10

Conjuntos

Colecciones **no ordenadas y sin duplicados**.

```
colores = {"rojo", "verde", "azul"}
```

Método	Descripción	Ejemplo
.add(x)	Añade un valor	colores.add("amarillo")
.remove(x)	Elimina un valor	colores.remove("rojo")
.union()	Unión de conjuntos	a.union(b)
.intersection()	Intersección	a.intersection(b)

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Diccionarios

Colecciones **no ordenadas** de pares **clave:valor**. Muy potentes y usados frecuentemente.

```
persona = {
    "nombre": "Ana",
    "edad": 30,
    "activo": True
}
```

Método	Descripción	Ejemplo
.get(clave)	Obtiene el valor sin error si no existe	persona.get("nombre")
.keys()	Devuelve las claves	persona.keys()
.values()	Devuelve los valores	persona.values()
.items()	Devuelve pares clave-valor	persona.items()
.update()	Actualiza el diccionario	persona.update({"edad": 31})
.pop(clave)	Elimina una clave	persona.pop("activo")

Acceso a elementos en diccionarios

Los diccionarios se acceden por **clave**, y la clave debe ir entre **comillas** si es una cadena:

```
persona = {"nombre": "Ana", "edad": 28}

print(persona["nombre"])  # → Ana
```

Siempre , siempre que la **clave sea un texto**, debe ir **entre comillas simples o dobles**:

```
persona['nombre']  # Correcto

persona["nombre"]  # También correcto
```

8.7 CADENAS DE TEXTO (STRINGS) EN PYTHON

Operadores con cadenas

Operador	Uso	Ejemplo	Resultado
+	Concatenación	"Hola" + " Mundo"	"Hola Mundo"
*****	Repetición	"Ha" * 3	"HaHaHa"
in	Comprobación de contenido	"a" in "python"	False
not in	Comprobación de exclusión	"z" not in "abc"	True

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Métodos útiles de cadenas

Método	Descripción	Ejemplo	Resultado
.lower()	Convierte a minúsculas	"Python".lower()	"python"
.upper()	Convierte a mayúsculas	"python".upper()	"PYTHON"
.replace(a, b)	Reemplaza texto	"Hola mundo".replace("Hola", "Hi")	"Hi mundo"
.split()	Divide la cadena por espacios o separador	"uno,dos,tres".split(",")	["uno", "dos", "tres"]
.strip()	Elimina espacios iniciales y finales	" hola ".strip()	"hola"
.find(sub)	Devuelve el índice de la subcadena o -1	"Python".find("th")	2

8.8 FRAMEWORKS EN PYTHON

Un **framework** es un conjunto de herramientas y librerías que facilita el desarrollo de aplicaciones. En Python existen frameworks especializados para web, ciencia de datos, automatización, interfaces gráficas y más.

Tipo de framework	Ejemplos destacados	Uso principal
Web (backend/API)	**Django, Flask**, FastAPI, Pyramid	Desarrollo de sitios web y APIs REST
Ciencia de datos / IA	**TensorFlow, scikit-learn**, PyTorch	Machine learning, modelos de IA, análisis
Visualización de datos	**matplotlib, seaborn**, Plotly	Gráficos, dashboards, visualización avanzada
Testing	**pytest, unittest**, nose	Pruebas automáticas de código
Interfaces gráficas	**Tkinter, PyQt**, Kivy	Aplicaciones de escritorio con GUI
Automatización / Scraping	**Selenium, Scrapy,** BeautifulSoup	Bots, extracción de datos, navegación web



Django es una de las preguntas más frecuentes en cuanto frameworks y Python se refiere

Frameworks para testing

- unittest: Módulo estándar para pruebas unitarias.
- pytest: Más potente y con sintaxis sencilla.
- doctest: Permite probar desde los comentarios del código.

Supera. by thePower © Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

9. LENGUAJE DE PROGRAMACIÓN RUBY

Ruby es un lenguaje de programación **interpretado, orientado a objetos y de alto nivel**. Fue creado por **Yukihiro "Matz" Matsumoto** y su primera versión apareció en 1995.

Ruby destaca por su **sintaxis limpia y expresiva**, que permite escribir código claro y fácil de mantener. Es especialmente conocido por el framework web **Ruby on Rails**, que revolucionó el desarrollo ágil de aplicaciones web.

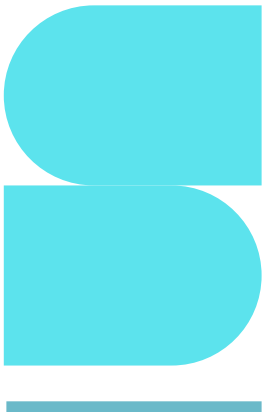
Característica	Descripción
Lenguaje completamente orientado a objetos	Todo es un objeto, incluso tipos primitivos.
Interpretado y dinámico	No requiere compilación previa, tipado dinámico.
Multiparadigma	Soporta POO, funcional, imperativa.
Multiplataforma	Funciona en Windows, Linux, macOS, etc.

9.1 FRAMEWORKS EN RUBY

Ruby es conocido por su enfoque limpio y orientado a objetos, y aunque no tiene tantos frameworks como Python, **ofrece potentes herramientas para el desarrollo rápido y elegante de aplicaciones**, especialmente web.

Framework	Descripción	Ideal para...
Ruby on Rails	El más popular. Framework, con ORM, rutas, vistas, generadores... Sigue el patrón MVC	Aplicaciones web completas y escalables
Sinatra	Microframework ligero y minimalista. Fácil de usar para APIs o apps pequeñas.	Prototipos rápidos, microservicios, APIs
Hanami	Modular, moderno y orientado a la arquitectura limpia.	Aplicaciones profesionales mantenibles
Padrino	Extensión de Sinatra con más herramientas (admin, internacionalización...).	Apps medianas que necesitan algo más que Sinatra
Cuba	Framework minimalista y rápido. Enfocado en rendimiento y simplicidad extrema.	APIs RESTful ligeras

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025.

Queda rigurosamente prohibida cualquier forma de reproducción, distribución comunicación pública o transformación total o parcial de esta obra sin el permiso escrito de los titulares de los derechos de explotación.